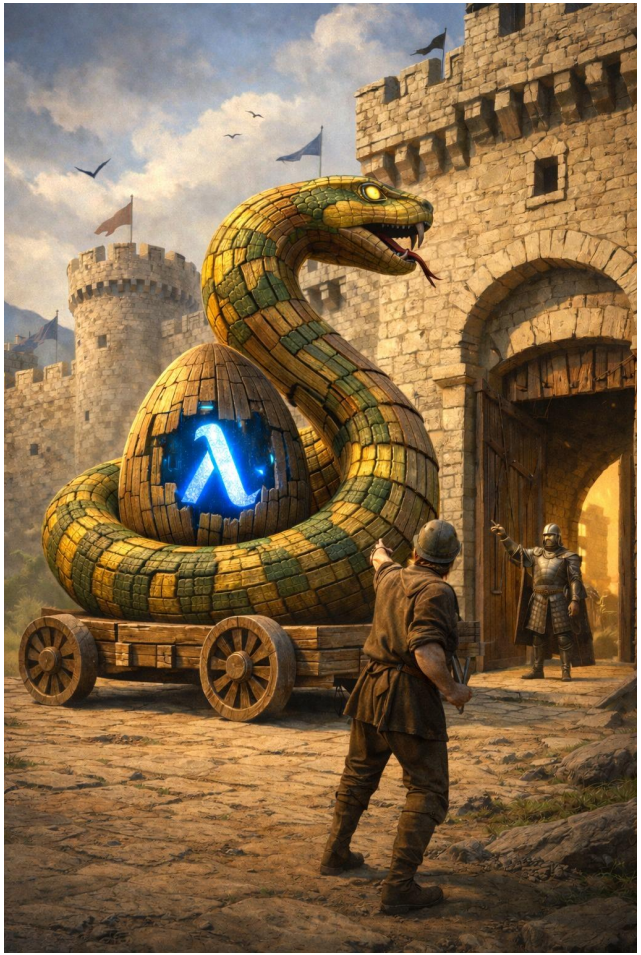
Programação funcional com Python

Augusto Rengel

Sumário

- O que é Programação Funcional?
- Fundamentos de FP
  - Funções como *first-class citizens*
  - Funções puras e impuras
  - Imutabilidade
  - Funções de ordem superior
  - Composição de funções
  - Aplicação parcial
- List Comprehensions e Generator Expressions
- Pipelines
- Limitações de FP em python
  - Contexto histórico
  - *Tail Call Optimization*
  - *Pattern matching*
  - Mutabilidade por padrão
  - Lambdas limitadas
  - Limitações de sintaxe
- Consequências das limitações
- Conclusão e Recomendações

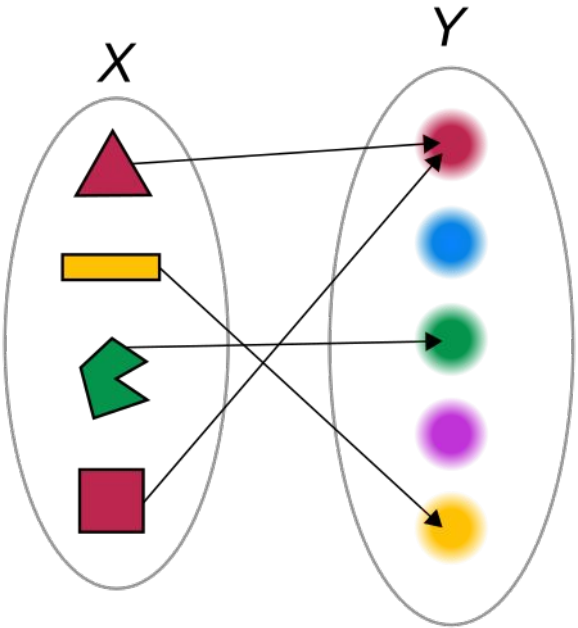


O que é Programação Funcional?

Programação funcional (FP) é um paradigma de programação em que programas são construídos aplicando e compondo funções.

É um paradigma declarativo, onde funções descrevem transformações de valores.

Ao invés de variáveis mutáveis, temos valores imutáveis com funções que realizam mapeamento de um conjunto para outro.



Fundamentos de FP

Funções como *first-class citizens*

Em Python, funções são *first-class citizens*, ou seja, são tratadas como qualquer outro valor.

Isso significa que podem ser:

- Atribuídas a variáveis

```python
def add(x: int, y: int) -> int:
    return x + y

f = add
f(2, 3)
```

```
5
```

Funções como *first-class citizens*

Em Python, funções são *first-class citizens*, ou seja, são tratadas como qualquer outro valor.

Isso significa que podem ser:

- Atribuídas a variáveis

- Passadas como argumento

```python
from typing import Callable


def apply(f: Callable[[int], int], x: int) -> int:
    return f(x)

apply(lambda x: x * 2, 5)
```

```
10
```

Funções como *first-class citizens*

Em Python, funções são *first-class citizens*, ou seja, são tratadas como qualquer outro valor.

Isso significa que podem ser:

- Atribuídas a variáveis

- Passadas como argumento

- Retornadas de outras funções

```python
from typing import Callable


def make_adder(n: int) -> Callable[[int], int]:
    def add(x: int):
        return x + n
    return add

add5 = make_adder(5)
add5(3)
```

```
8
```

Funções puras e impuras

Funções puras sempre retornam o mesmo resultado para a mesma entrada e não causam efeitos colaterais, enquanto funções impuras dependem ou modificam estado externo.

```python
# Pure function
def add(a: int, b: int) -> int:
    return a + b

print(add(2, 2))
print(add(2, 2))
```

```
4
4
```

```python
total: int = 0

def add_to_total(x: int) -> int:
    global total
    total += x
    return total

print(add_to_total(2))
print(add_to_total(2))
```

```
2
4
```

Imutabilidade

Em FP, trabalhamos com valores não modificáveis e não com variáveis.

```python
from typing import List, TypeVar

T = TypeVar("T")

def add_item(lst: List[T], item: T) -> List[T]:
    return lst + [item] # returns a new list

items = [1, 2, 3]

new_items = add_item(items, 4)

items.append(5)
print(new_items) # [1, 2, 3, 4]
```

```
[1, 2, 3, 4]
```

Imutabilidade

Isso difere de uma abordagem mutável, como a da imagem a seguir:

```python
from typing import List, TypeVar

T = TypeVar("T")

def add_item(lst: List[T], item: T) -> List[T]:
    lst.append(item)
    return lst

items = [1, 2, 3]

new_items = add_item(items, 4) # both items and ne

items.append(5)
print(new_items) # [1, 2, 3, 4, 5]
```

```
[1, 2, 3, 4, 5]
```

Funções de ordem superior

Funções de ordem superior são funções que recebem ou retornam outras funções.

```python
from typing import Callable, TypeVar

T = TypeVar("T")
U = TypeVar("U")

def apply(f: Callable[[T], U], x: T) -> U:
    return f(x)

apply(lambda x: x * 2, 5)
```

```
10
```

Funções de ordem superior

Algumas das funções de ordem superior mais comuns já estão nativamente disponíveis, alguns exemplos são:

```
list(map(lambda x: x * 2, [1, 2, 3]))
```

```
[2, 4, 6]
```

```
list(filter(lambda x: x % 2 == 1, [1, 2, 3, 4, 5]))
```

```
[1, 3, 5]
```

```
from functools import reduce

reduce(lambda x, y: x + y, [1, 2, 3, 4, 5]) # ((((1 + 2) + 3) + 4) + 5).
```

```
15
```

Composição de funções

```python
def double(x: int) -> int:
    return x * 2

def increment(x: int) -> int:
    return x + 1

def compose(f: Callable[[int], int], g: Callable[[int], int]) -> Callable[[int], int]:
    return lambda x: f(g(x))

print(f"double(increment(3)) = {compose(double, increment)(3)}") # 8
print(f"increment(double(3)) = {compose(increment, double)(3)}") # 7
```

```
double(increment(3)) = 8
increment(double(3)) = 7
```

Aplicação parcial

É quando passamos para uma função parte de seus argumentos e geramos uma nova função que precisa apenas dos argumentos restantes para ser executada.

```python
from functools import partial

def multiply(a: int, b: int) -> int:
    return a * b

double = partial(multiply, 2)

print(double(3)) # 6
print(double(5)) # 10
```

```
6
10
```

Aplicação parcial

É quando passamos para uma função parte de seus argumentos e geramos uma nova função que precisa apenas dos argumentos restantes para ser executada.

```
from toolz import curry   # type

@curry
def add(a: int, b: int) -> int:
    return a + b

add5 = add(5)
add5(3) # type: ignore
```

```
8
```

List Comprehensions e Generator Expressions

List comprehensions são uma ideia funcional, mas com sintaxe adaptada ao estilo Python.

É a aplicação de uma transformação combinada com um filtro em uma coleção.

A inspiração vem de Haskell:

```
[x * 2 | x <- [1,2,3], x > 0]
```

List Comprehensions e Generator Expressions

List comprehensions são uma ideia funcional, mas com sintaxe adaptada ao estilo Python.

É a aplicação de uma transformação combinada com um filtro em uma coleção.

```
result = [x * 2 for x in [1, 2, 3] if x > 0]

print(f"type={type(result)} | {result=}")
```

```
type=<class 'list'> | result=[2, 4, 6]
```

List Comprehensions e Generator Expressions

List comprehensions são uma ideia funcional, mas com sintaxe adaptada ao estilo Python.

É a aplicação de uma transformação combinada com um filtro em uma coleção.

```
result = (x * 2 for x in [1, 2, 3] if x > 0) # lazy evaluated, each value is yield

print(f"type={type(result)} | {result=}")
print(next(result))
print(list(result))
```

```
type=<class 'generator'> | result=<generator object <genexpr> at 0x73eb3fdce400>
2
[4, 6]
```

Pipelines

Pipelines

Pipelines são uma maneira de se aplicar uma sequência de transformações em um conjunto de dados.

Ao invés de chamar funções aninhadas:

$$f(g(h(x)))$$

Escrevemos transformações como um fluxo linear e legível:

$$x \rightarrow h \rightarrow g \rightarrow f$$

"Pipeline" imperativo

No estilo imperativo, transformações são aplicadas sequencialmente, detalhando como isso deve ser feito:

```python
from typing import List

data = [1, -2, 3, 0, 4]

filtered: List[int] = []
for x in data:
    if x > 0:
        filtered.append(x)

doubled: List[int] = []
for x in filtered:
    doubled.append(x * 2)

doubled
```

```
[2, 6, 8]
```

Pipelines

Normalmente em python usamos list comprehension, devido a esse ser um caso simples de filter -> map:

```
data = [1, -2, 3, 0, 4]

result = [x * 2 for x in data if x > 0]
result
```

```
[2, 6, 8]
```

Pipelines

E se o conjunto de dados for grande o bastante para precisarmos de maior eficiência, podemos utilizar generator expressions:

```
data = [1, -2, 3, 0, 4]

pipeline = (x for x in data if x > 0)
pipeline = (x * 2 for x in pipeline)

list(pipeline)
```

```
[2, 6, 8]
```

Pipelines

Podemos também usar libs como toolz:

```python
from toolz import pipe # type: ignore

data = [1, -2, 3, 0, 4]

result = pipe( # type: ignore
    data,
    lambda xs: filter(lambda x: x > 0, xs), # type: ignore
    lambda xs: map(lambda x: x * 2, xs), # type: ignore
    list
)

result
```

```
[2, 6, 8]
```

Pipelines

Podemos também usar libs como returns:

```python
from returns.pipeline import flow

data = [1, -2, 3, 0, 4]

result = flow( # type: ignore
    data,
    lambda xs: [x for x in xs if x > 0], # type: ignore
    lambda xs: [x * 2 for x in xs], # type: ignore
)

result
```

```
[2, 6, 8]
```

Pipelines

O que também pode ser aproveitado em libs famosas como pandas:

```python
import pandas as pd

df = pd.DataFrame({
    "value": [1, -2, 3, 0, 4]
})

(df
 .pipe(lambda d: d[d["value"] > 0])
 .pipe(lambda d: d.assign(value=d["value"] * 2))
)
```

	value
0	2
2	6
4	8

Limitações de FP em python

Contexto Histórico

Guido, o criador de python, resume bem como python se relaciona com FP em um post de 2009: Origins of Python's "Functional" Features

> "*I didn't view Python as a functional programming language. However, earlier on, it was clear that users wanted to do much more with lists and functions.*"

Contexto Histórico

Guido, o criador de python, resume bem como python se relaciona com FP em um post de 2009: Origins of Python's "Functional" Features

> "*Lambda was really only intended to be a syntactic tool for defining anonymous functions. However, the choice of terminology had many unintended consequences. For instance, users familiar with functional languages expected the semantics of lambda to match that of other languages. As a result, they found Python's implementation to be sorely lacking in advanced features.*"

Contexto Histórico

Guido, o criador de python, resume bem como python se relaciona com FP em um post de 2009: Origins of Python's "Functional" Features

> "*Curiously, the map, filter, and reduce functions that originally motivated the introduction of lambda and other functional features have to a large extent been superseded by list comprehensions and generator expressions. In fact, the reduce function was removed from list of builtin functions in Python 3.0.*"

Contexto Histórico

Guido, o criador de python, resume bem como python se relaciona com FP em um post de 2009: Origins of Python's "Functional" Features

> "*Lastly, even though a number of functional programming features have been introduced over the years, Python still lacks certain features found in “real” functional programming languages.*"

Contexto Histórico

Guido, o criador de python, resume bem como python se relaciona com FP em um post de 2009: Origins of Python's "Functional" Features

Em outras palavras, python tem recursos de programação funcional, mas não é uma linguagem funcional e isso é intencional.

Tail Call Optimization

Python não faz otimização de chamadas recursivas (tail call optimization).

O criador da linguagem cita como uma das justificativas manter o stack trace:

Tail Recursion Elimination - 2009

Pattern matching (não exaustivo)

Python tem *match*, mas não tem verificação de exaustividade.

Algo que pode ser feito para tentar se aproximar desse comportamento é alterar a configuração *typeCheckingMode* do *pyright* para o modo *strict*, com isso passamos a ver um warning em matchs incompletos.

Pattern matching (não exaustivo)

```python
from typing import Literal

Status = Literal["success", "error", "canceled"]

# if typeCheckingMode is set to strict in your linter you should see the error:
# Cases within match statement do not exhaustively handle all values
# Unhandled type: "Literal['canceled']"
def handle(status: Status):
    match status:    Cases within match statement do not exhaustively handle all
        case "success":
            return 1
        case "error":
            return 0

print(handle("success"))
print(handle("canceled"))
```

```
1
None
```

Mutabilidade por padrão

Apesar de alguns tipos serem imutáveis por design:

- int

- float

- str

- tuple

Em Python, imutabilidade é uma convenção, não uma garantia.

```
n = 2
m = n
m += 2

print(f"{m=} e {n=}")
```

```
m=4 e n=2
```

Mutabilidade por padrão

É dito que é uma convenção porque mesmo que existam maneiras de se criar "tipos" imutáveis como:

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class User:
    name: str
    age: int

u = User("Olivia", 30)
print(u)
# u.age = 31 # resulta em um erro em runtime
```

```
User(name='Olivia', age=30)
```

Mutabilidade por padrão

A linguagem ainda permite burlar isso:

```
object.__setattr__(u, 'age', 31)
print(u)
```

```
User(name='Olivia', age=31)
```

Lambdas limitadas

Lambdas em Python são restritas a uma única expressão.

```
lambda x:
    if x < 0:
        return 0
    y = x * 2
    return y + 1
```

```
def process(x: int):
    if x < 0:
        return 0
    y = x * 2
    return y + 1
```

Lambdas limitadas

Com a adição de operadores ternarios passamos a poder fazer algo assim, mas não é recomendado por não ser "pythonico":

```
lambda x: 0 if x < 0 else (x * 2) + 1
```

Limitações de sintaxe

Python não possui operadores nativos para composição ou pipeline.

Portanto não podemos escrever algumas operações comuns de modo simplificado:

```
h = f >> g
```

ou

```
h =
    dados
    |> filtrar
    |> transformar
    |> validar
```

Consequências das limitações

Consequências das limitações

FP ainda é relevante para python e é comum ver seu uso localizado, na forma de:

- list comprehensions

- generator expressions

- funções tratadas como *first-class citizens*

Mas implementações globais são raras, devido a falta de adesão completa ao paradigma funcional por parte da linguagem.

Conclusão e Recomendações

Conclusão e Recomendações

- Foque no uso de funções puras, construindo funções menores e objetivas. Com isso não será necessário replicar o estado do sistema para executar testes, basta pensar em termos de entradas e saídas.
- Use *mypy* para adicionar verificações de tipos ao seu código e priorize a redução de mutabilidade.
- Escreva funções que possam ser compostas com facilidade, seja por meio de decoradores, pipelines ou encadeamento.
- Prefira classes imutáveis, utilizando *dataclasses* ou *BaseModel* (*Pydantic*).
- Use FP para implementar as regras de negócio e modelar o domínio de suas aplicações. O livro Domain Modeling Made Functional é uma ótima referência para isso.

Conclusão e Recomendações

- Explore também os módulos nativos voltados a FP, listados na documentação oficial
- Há ainda algumas bibliotecas úteis feitas pela comunidade:
  - Expression
  - Toolz
  - Returns

Conclusão e Recomendações

- Leituras recomendadas:
  - Functional Programming HOWTO
  - Functional Programming In Python - David Mertz
  - Domain Modeling Made Functional - Scott Wlaschin

Obrigado!